
SSM Optimization

Weekly Meeting

12/06/2026

Speculative decoding for SSMs

- Main challenges:
 1. **Parallel mode** for **verification** is not very efficient, as it is tuned for very long sequences;
 2. **Caching states** for possible backtracking is expensive.
- There are mainly three works considering SpecDec for SSMs (only one considers also quantization).

Efficient verification

[R1] deals with the verification issue by implementing a fused multi-step scan-update kernel that efficiently processes a short sequence of K tokens in one pass.

seqLen	BS=1			BS=4			BS=16		
	SSU	CS	Ours	SSU	CS	Ours	SSU	CS	Ours
1	0.0074	0.064	0.0074	0.044	0.22	0.044	0.18	0.76	0.18
5	0.027	0.066	0.016	0.23	0.22	0.055	0.89	0.77	0.20
10	0.052	0.067	0.027	0.46	0.22	0.086	1.78	0.79	0.31

Table 1: Run-time (in *ms*) of different state update kernels under different batch-size (BS) and multi-token forward sequence length (seqLen). SSU stands for Selective-Scan kernel; CS for Chunk-Scan kernel; Ours for Fused Selective-Scan kernel. Since Selective-Scan kernel only takes input of sequence length 1, we use a for-loop to iterate through the sequence length.

Algorithm 1 Fused multi-step State-Update kernel. This Algorithm is simplified with the only input being x, A, B, C, Δ, h in reference to the implementation of [10].

Notation $Z^* \leftarrow Z$: load Z into shared memory for computation. $Z \leftarrow Z^*$: Storing Z out to High-Bandwidth-Memory (HBM). Any variable with $*$ means that this variable is in shared memory, everything else is in HBM.

Shapes B - Batch, L - Sequence Length, H - Num heads, P - State Dimension, D - Head Dimension, G - Num groups

Input: $h_0 : (B, H, D, P)$ - initial state, $x : (B, L, H, D)$ $A : (H, D, P)$, $B : (B, L, G, P)$, $C : (B, L, G, P)$, $\Delta : (B, L, H, D)$

Output: $y : (B, L, H, D)$ - Output, $\hat{h} : (B, H, D, P)$ - updated state

```

1:  $h^* \leftarrow h_0$ 
2:  $i \leftarrow 0$ 
3: while  $i < L$  do
4:    $x^* \leftarrow x[:, i, :, :]$ 
5:    $\Delta^* \leftarrow \Delta[:, i, :, :]$ 
6:    $B^* \leftarrow B[:, i, :, :]$ 
7:    $C^* \leftarrow C[:, i, :, :]$ 
8:    $A^* \leftarrow A$ 
9:    $\bar{A}^* \leftarrow \exp(\Delta^* \times A^*)$ 
10:   $\bar{B}^* \leftarrow \Delta^* \times B^*$ 
11:   $h^* \leftarrow h^* \times \bar{A}^* + \bar{B}^* \times x^*$ 
12:   $y^* \leftarrow C^* \times h^*$ 
13:   $y[:, i, :, :] \leftarrow y^*$ 
14: end while
15:  $\hat{h} \leftarrow h^*$ 

```

▷ State is not stored out to HBM at this point

[R1] Wu et al., “Snakes and Ladders: Accelerating State Space Model Inference with Speculative Decoding”, NeurIPS Workshop 2024

Efficient verification

[R2] also deals with the verification issue implementing a multi-step scan-update kernel:

$$y_{j:k}, \mathbf{h}_j, \mathbf{h}_k \leftarrow \text{MULTISTEP}(\mathbf{h}_i, y_{1:n}, i, j, k; \mathbf{A}, \mathbf{B}, \mathbf{C}, \Delta)$$

Last verified state,
before draft tokens

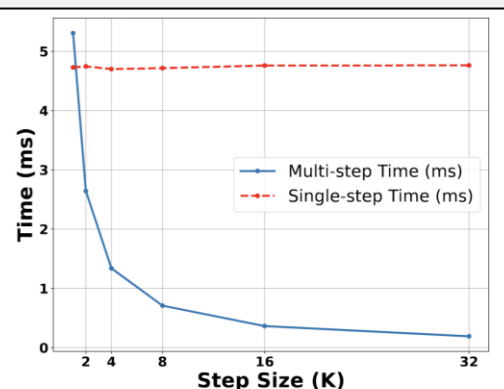


Figure 3: Performance of the multi-step SSM kernel for generating 32 tokens.

Algorithm 2 Multi-Step Linear RNN Speculation

```

function VERIFY( $y_{1:k}, j, \mathbf{h}_i$ )
    //  $y_{1:k}$  are draft,  $j$  is last verified,
    //  $\mathbf{h}_i$  is a cached state with  $i \leq j$ 
     $y'_{j:k}, \mathbf{h}_j, \mathbf{h}_k \leftarrow$ 
    MULTISTEP( $\mathbf{h}_i, y_{1:k}, i, j, k; \theta_v$ )
     $k' \leftarrow \text{FIRSTCONFLICT}(y_{j:k}, y'_{j:k})$ 
    return  $k', \mathbf{h}_k$  if  $k' = k$  else  $\mathbf{h}_j$ 

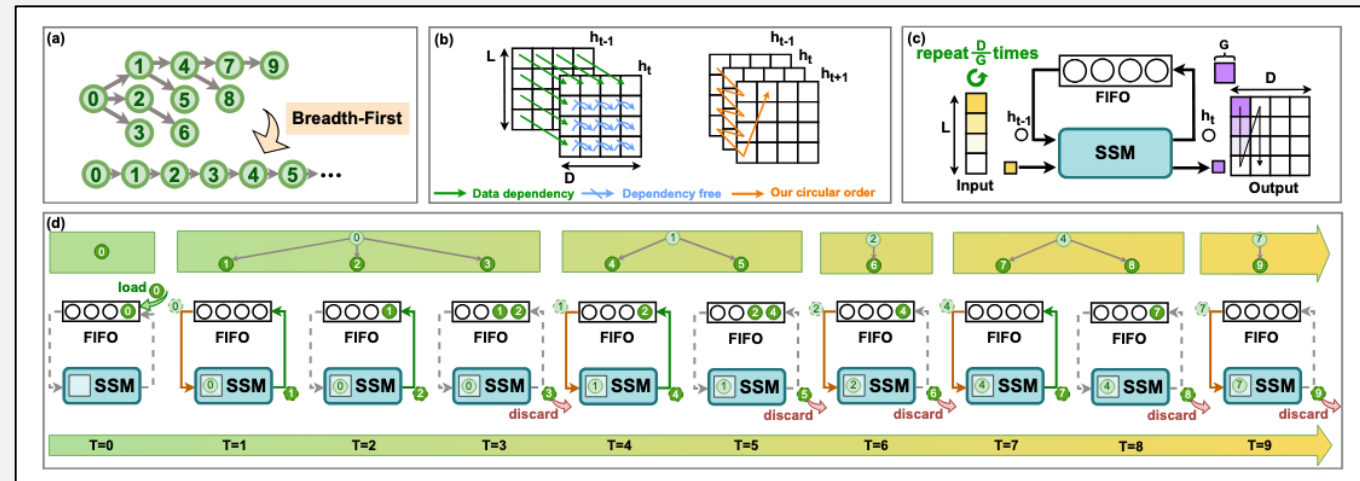
function SPECULATE( $K$ )
    //  $K$  tokens are drafted per step
     $\mathbf{h}_{\text{cache}} \leftarrow \mathbf{h}_0$ 
     $j \leftarrow 0$ 
    while  $y_j$  is not end do
         $k \leftarrow j + K$ 
         $y_{j+1:k} \leftarrow \arg \max p(y_{j+1:k} \mid y_{1:j}, \theta_D)$ 
         $j, \mathbf{h}_{\text{cache}} \leftarrow \text{VERIFY}(y_{1:k}, j, \mathbf{h}_{\text{cache}})$ 
    return  $y_{1:j}$ 
  
```

[R2] Wang et al., “The Mamba in the Llama: Distilling and Accelerating Hybrid Models”, NeurIPS Workshop 2024

Efficient verification

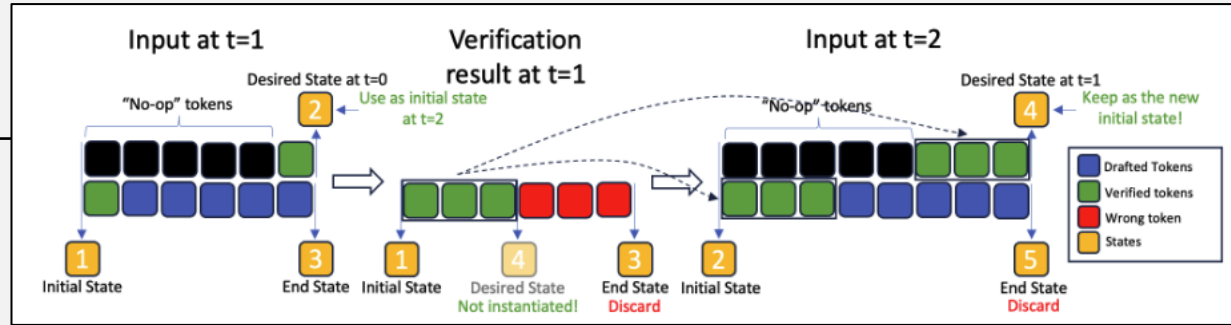
[R3] considers tree-structured draft verification. They employ breadth-first traversal and tile-wise processing.

- They cache states with the same parent nodes.
- Computation is done tile-wise, as the reduction in memory overhead and the ability to keep all processing on-chip outweigh the extra iterations.



[R3] Zhong et al., "SpecMamba: Accelerating Mamba Inference on FPGA with Speculative Decoding", ICCAD 2025

State caching



[R1] deals with the state backtracking with two alternative approaches:

1. **Joint attainment and advancement:** at each verification timestep, add an extra sequence to the drafts batch with the verified tokens at the previous timestep, left-padded with special “no operation” tokens. The backtracking sequence is used to update the global SSM state, while the other sequences are used for speculative verification of new tokens.
2. **Activation replay:** when verifying a batch of speculated tokens, ARP stores the inputs to the SSM block (smaller than the state) in a cache, with which ARP recomputes the SSM state up to the last correct token.

[R1] Wu et al., “*Snakes and Ladders: Accelerating State Space Model Inference with Speculative Decoding*”, NeurIPS Workshop 2024

State caching

[R2] saves a snapshot of the state before evaluating the draft tokens. This allows recomputing the correct state on the fly after a token is rejected.

- Efficiency guaranteed by the multi-step kernel

[R2] Wang et al., “*The Mamba in the Llama: Distilling and Accelerating Hybrid Models*”, NeurIPS Workshop 2024

State caching

[R3] considers a hybrid approach:

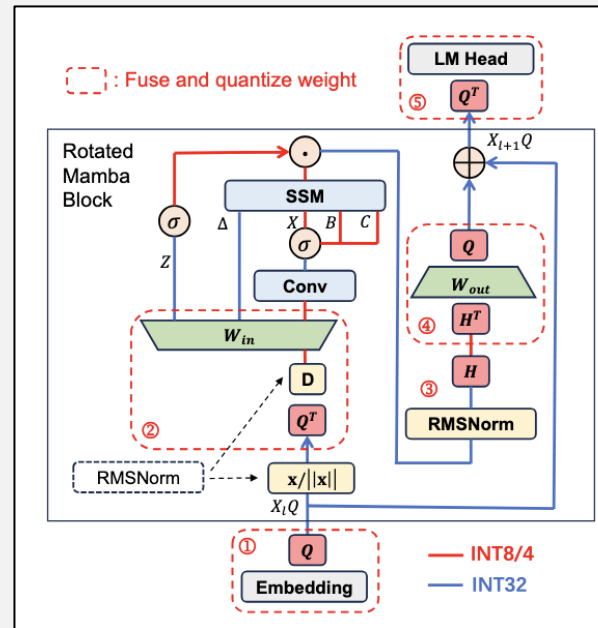
- Off-chip state storage for draft model;
- On-chip activation caching for target model.

If some tokens are rejected, the draft model resumes from the last accepted hidden state (loaded from off-chip), and the target model recomputes the necessary hidden states using the cached activations.

[R3] Zhong et al., “*SpecMamba: Accelerating Mamba Inference on FPGA with Speculative Decoding*”, ICCAD 2025

SpecDec + quantization

[R3] also quantizes all models to INT4 precision following a prior work [R4].



[R3] Zhong et al., "SpecMamba: Accelerating Mamba Inference on FPGA with Speculative Decoding", ICCAD 2025

[R4] Wei et al., "LightMamba: Efficient Mamba Acceleration on FPGA with Quantization and Hardware Co-design", DATE 2025

Results

[R1]

Inference strategy	MT-Bench	GSM-8K	HumanEval
Baseline	70.36	70.37	70.26
Joint + PNG(6,2)	112.67 (1.60×)	137.49 (1.95×)	133.64 (1.90×)
ARP + PNG(6,2)	103.80 (1.47×)	122.73 (1.74×)	120.66 (1.72×)

Table 5: Inference speed (in tokens/sec.) of our framework using a simple prompt n -gram ($n = 6$, $d = 2$ drafts) draft mechanism with Mamba-2 on 1xA10G Nvidia GPU.

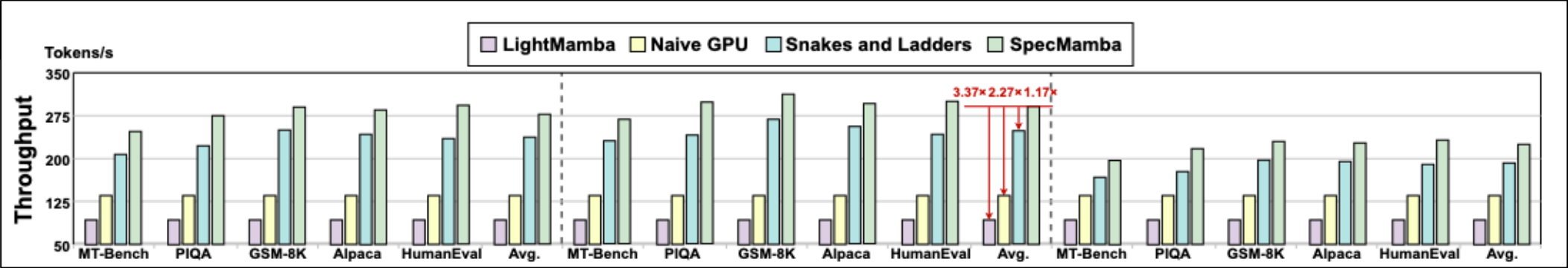
[R2]

Model Size	GPU	K	# Gen. Tokens	Throughput (toks/s)	Speedup
2.8B	3090	3	3.01	259	2.3x
2.8B	3090	4	3.28	289	2.6x
2.8B	H100	3	4.04	389	1.71x
2.8B	H100	4	3.9	421	1.85x
7B	3090	3	3.19	109	2.1x
7B	3090	4	3.56	110	2.1x
7B	H100	3	3.28	271	1.95x
7B	H100	4	3.6	272	2x

Table 1: Speedup results for speculative decoding with pure Mamba models. The 2.8B verifier uses a 130M Mamba draft. The 7B verifier uses a Llama3 1B draft we trained. Data is from The Pile. K is number of draft tokens produced, # Gen includes an additional token from the last verifier logits.

Results

[R3]



Next steps

- Proceed to quantize activations (state in particular could be interesting), to try to overcome the memory-bound regime
 - Start with RTN.
 - Analyze sensitivity.
- Think about integration with SpecDec approaches.